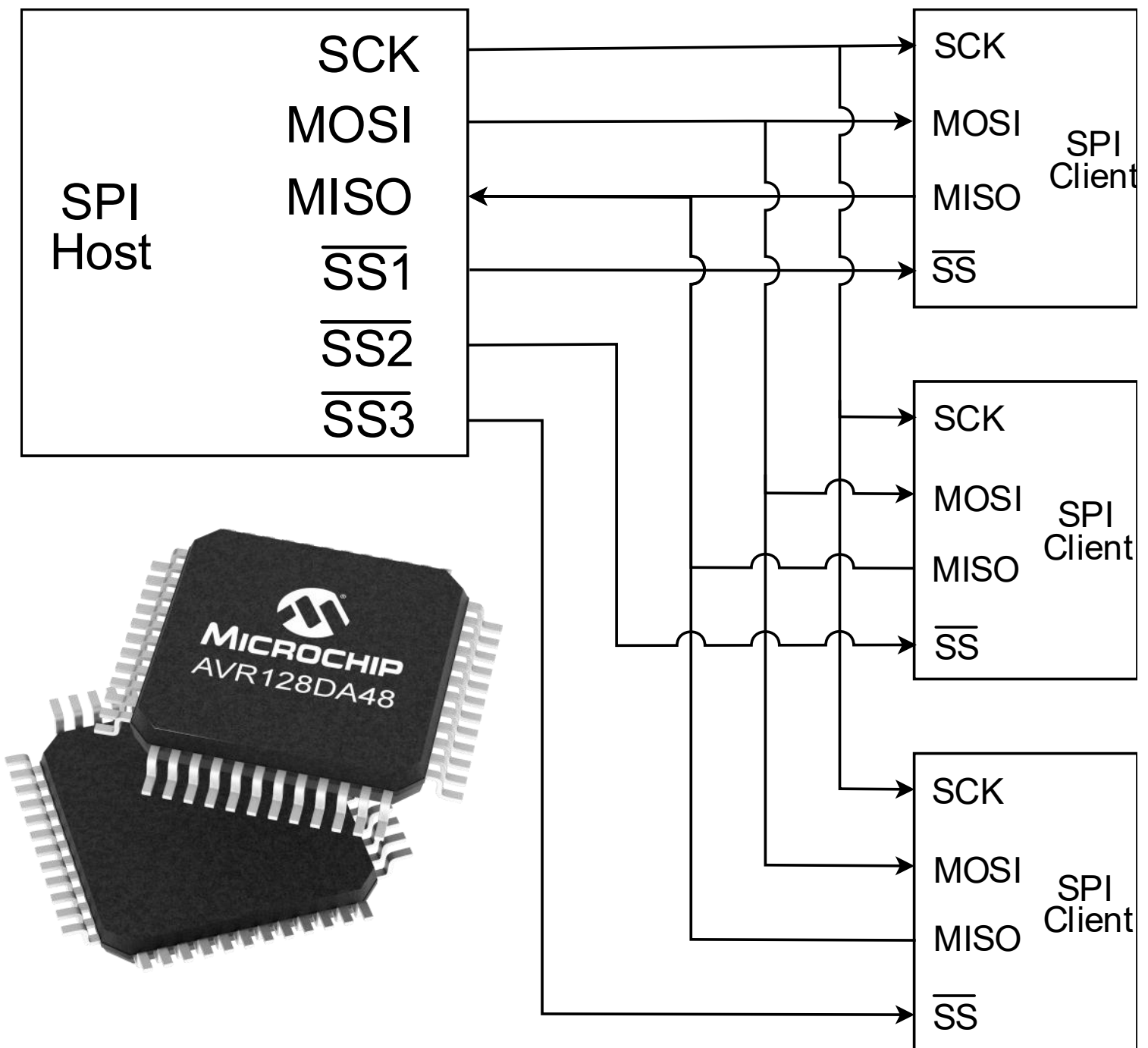


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 14 - SPI Architecture Patterns



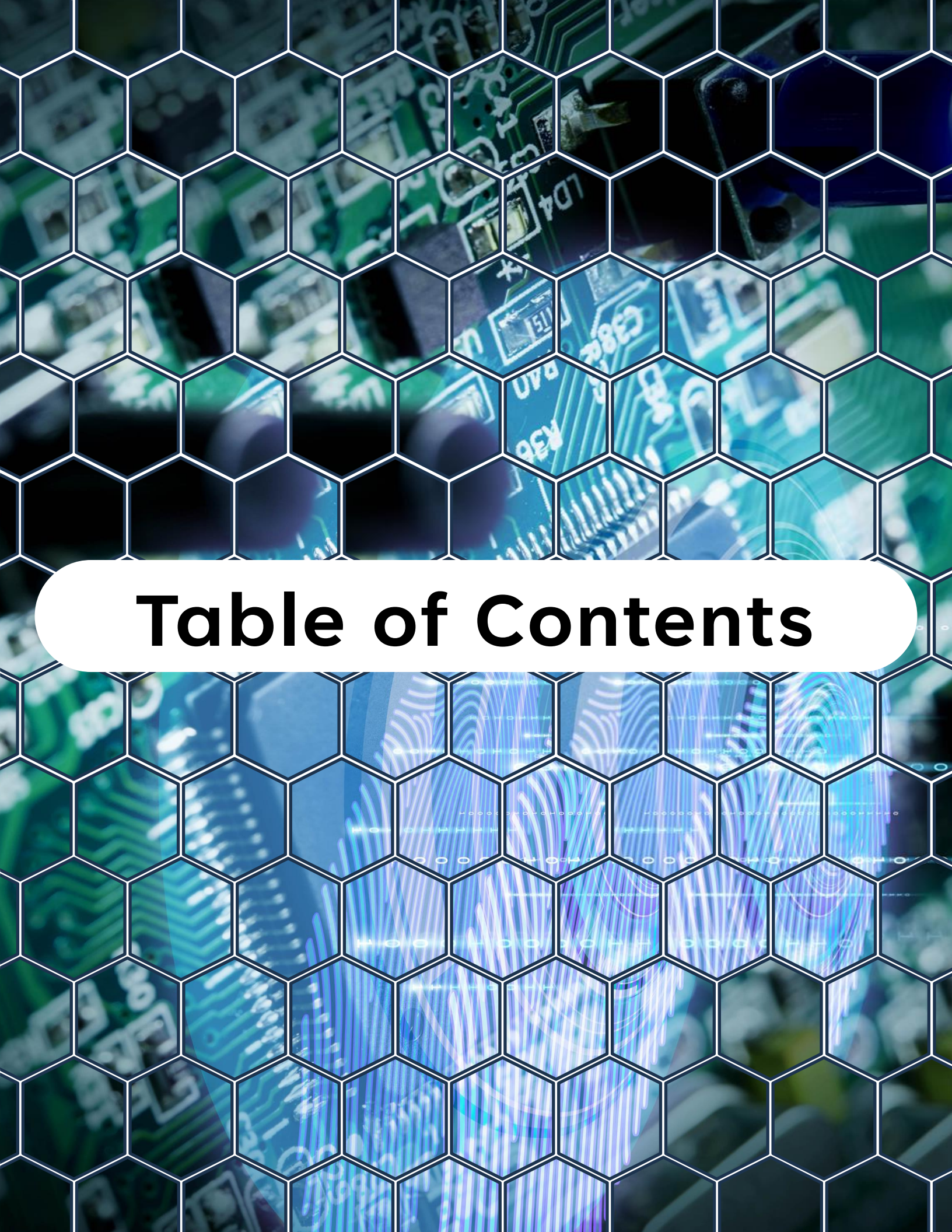
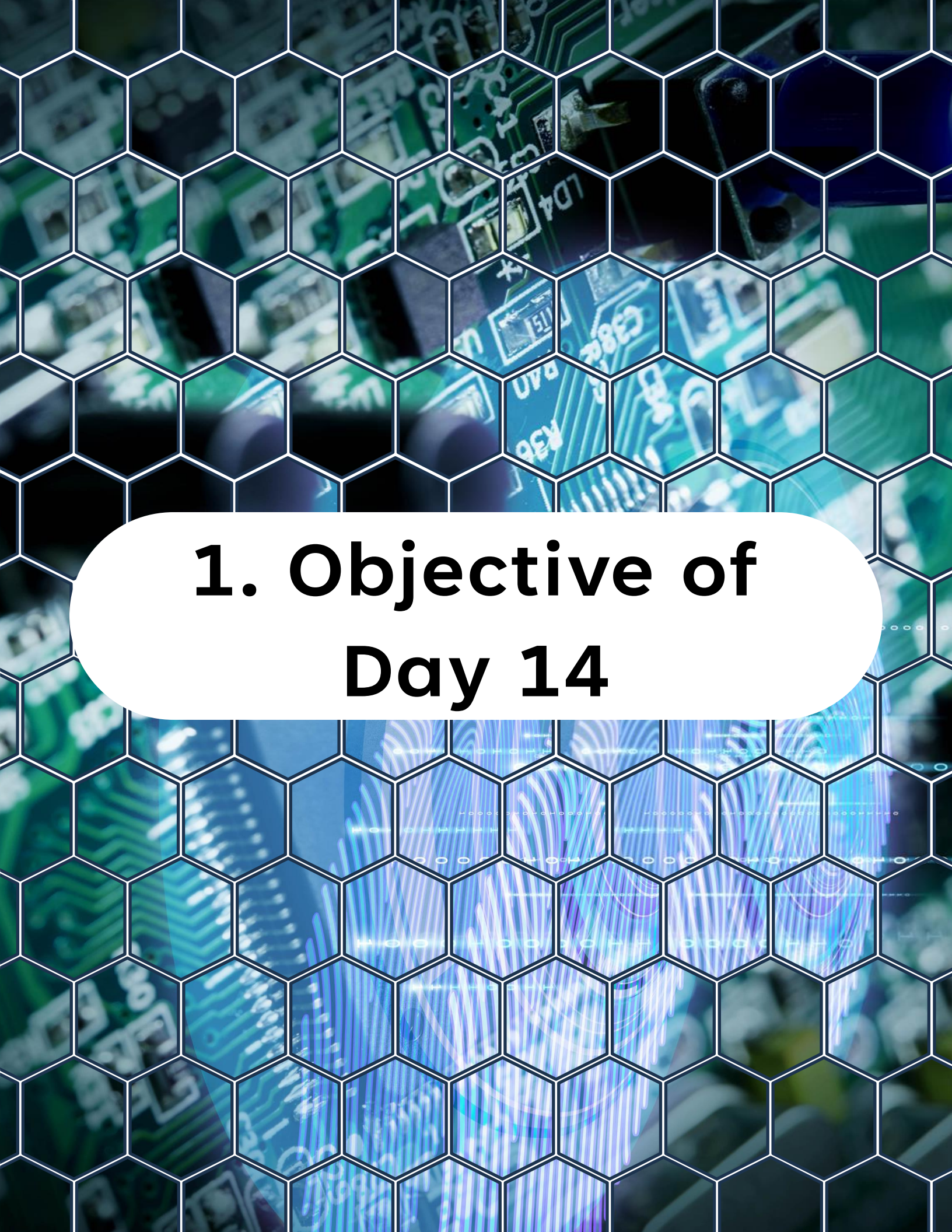


Table of Contents

Table of Contents

1. Objective of Day 14
2. Why Architecture Matters
3. Layered SPI Architecture
4. Designing a Minimal SPI Driver API
5. Blocking vs Non-Blocking SPI
6. Adding Timeout Protection
7. SPI Transaction Pattern
8. State Machine Pattern for Non-Blocking SPI
9. Example - Simple SPI Flash Command Driver
10. Handling Multiple SPI Devices
11. Error Handling Strategy
12. Debugging SPI Architecture Issues
13. What You Learned Today
14. What Comes Next



1. Objective of Day 14

1. Objective of Day 14

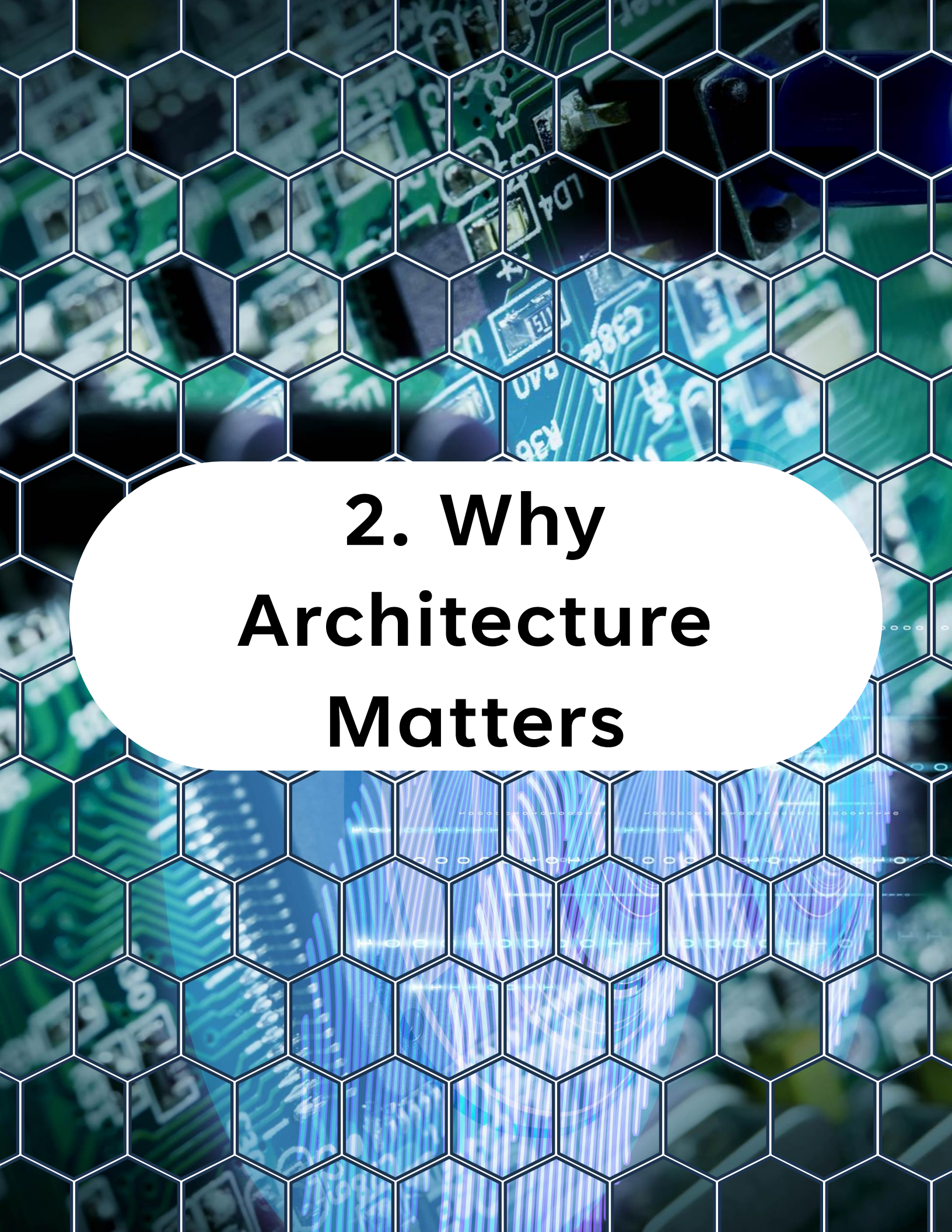
Yesterday, you built a working SPI Master using blocking transfers.

Today you move from “it works” to “it scales.”

By the end of this day, you will:

- Separate SPI driver from application logic
- Design reusable SPI APIs
- Add timeout protection
- Understand blocking vs non-blocking SPI patterns
- Build a transaction-based SPI architecture
- Prepare for real devices like sensors, flash, and displays

This is where firmware design maturity begins.



2. Why Architecture Matters

2. Why Architecture Matters

A quick demo SPI function like this:



```
1 spi0_transfer_byte(0xAA);
```

is fine for testing.

But real projects quickly grow:

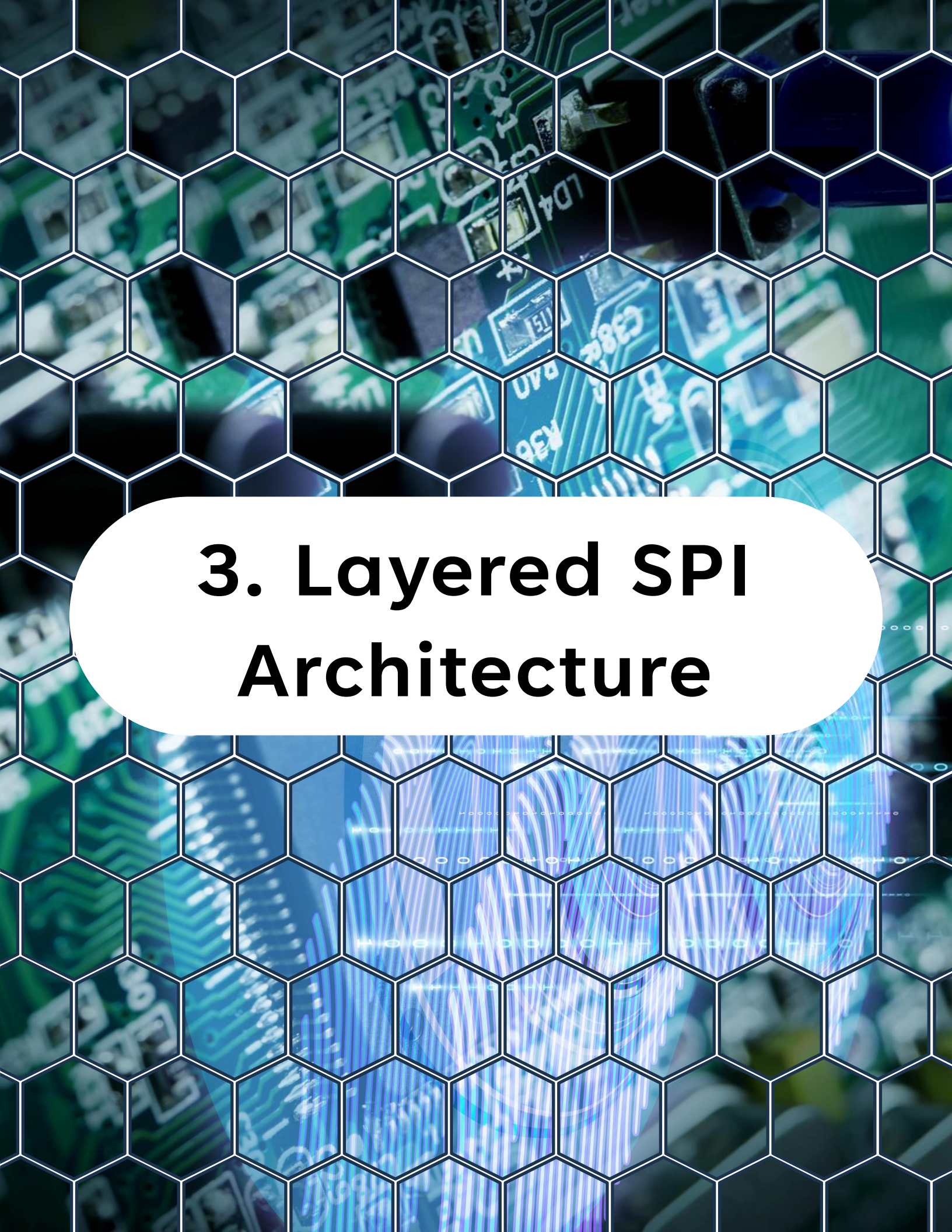
- Multiple SPI slaves
- Different SPI modes
- Timeouts required
- Large transfers
- Interrupt-based operation
- Error handling

2. Why Architecture Matters

If you mix hardware register code directly inside application logic, your firmware becomes fragile.

Professional firmware separates:

- Low-level driver
- Hardware abstraction
- Device-specific logic
- Application logic



3. Layered SPI Architecture

3. Layered SPI Architecture

A clean SPI design has layers:

Layer 1 - Hardware Driver

- Direct register access
- SPI enable/disable
- Transfer primitive
- Chip select control

Layer 2 - Transaction Layer

- Begin transaction
- Transfer buffer
- End transaction
- Timeout protection

3. Layered SPI Architecture

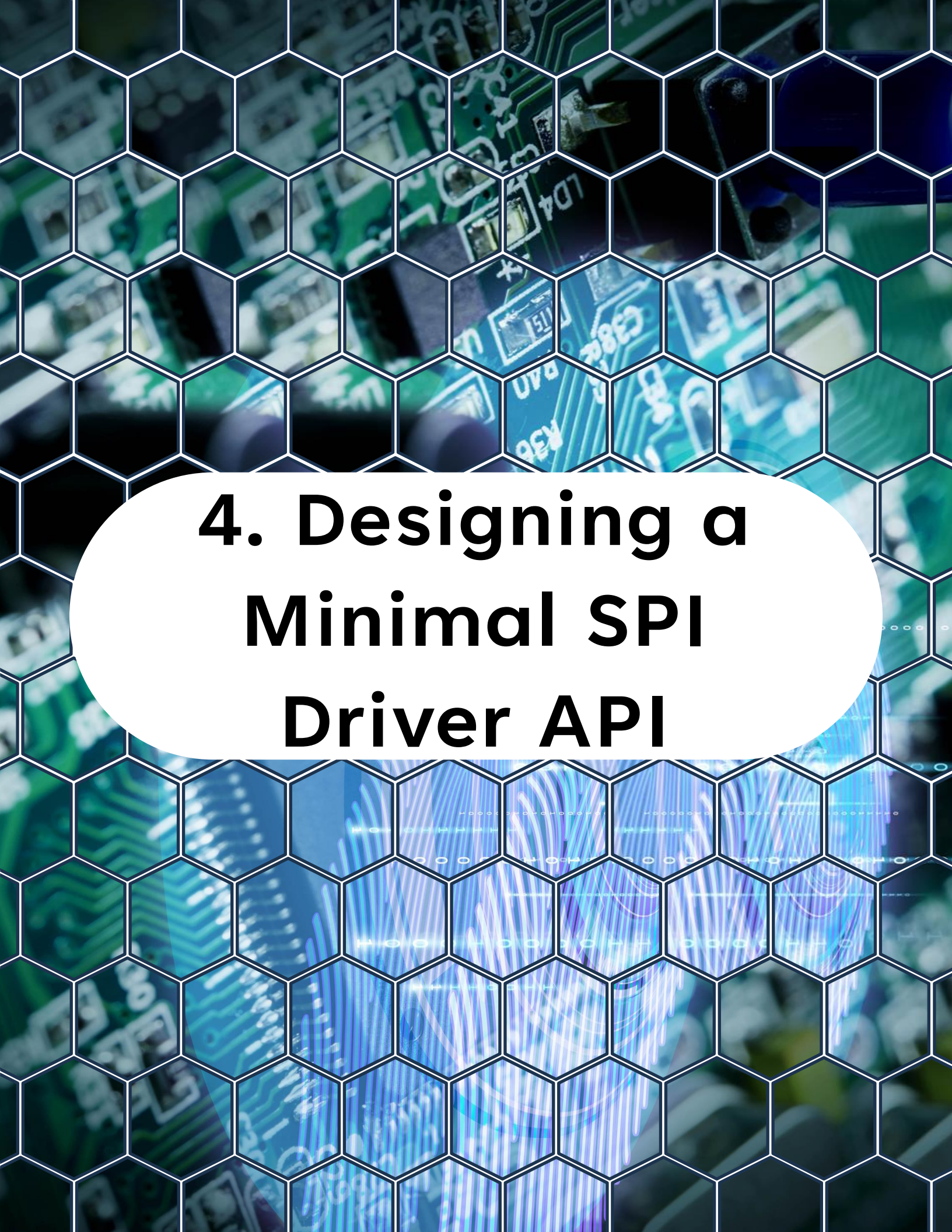
Layer 3 - Device Driver

- `Sensor_read_register()`
- `Flash_write_page()`
- `Display_send_command()`

Layer 4 – Application

- Uses device drivers
- No direct register access

This separation keeps your firmware maintainable.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in bold black font.

4. Designing a Minimal SPI Driver API

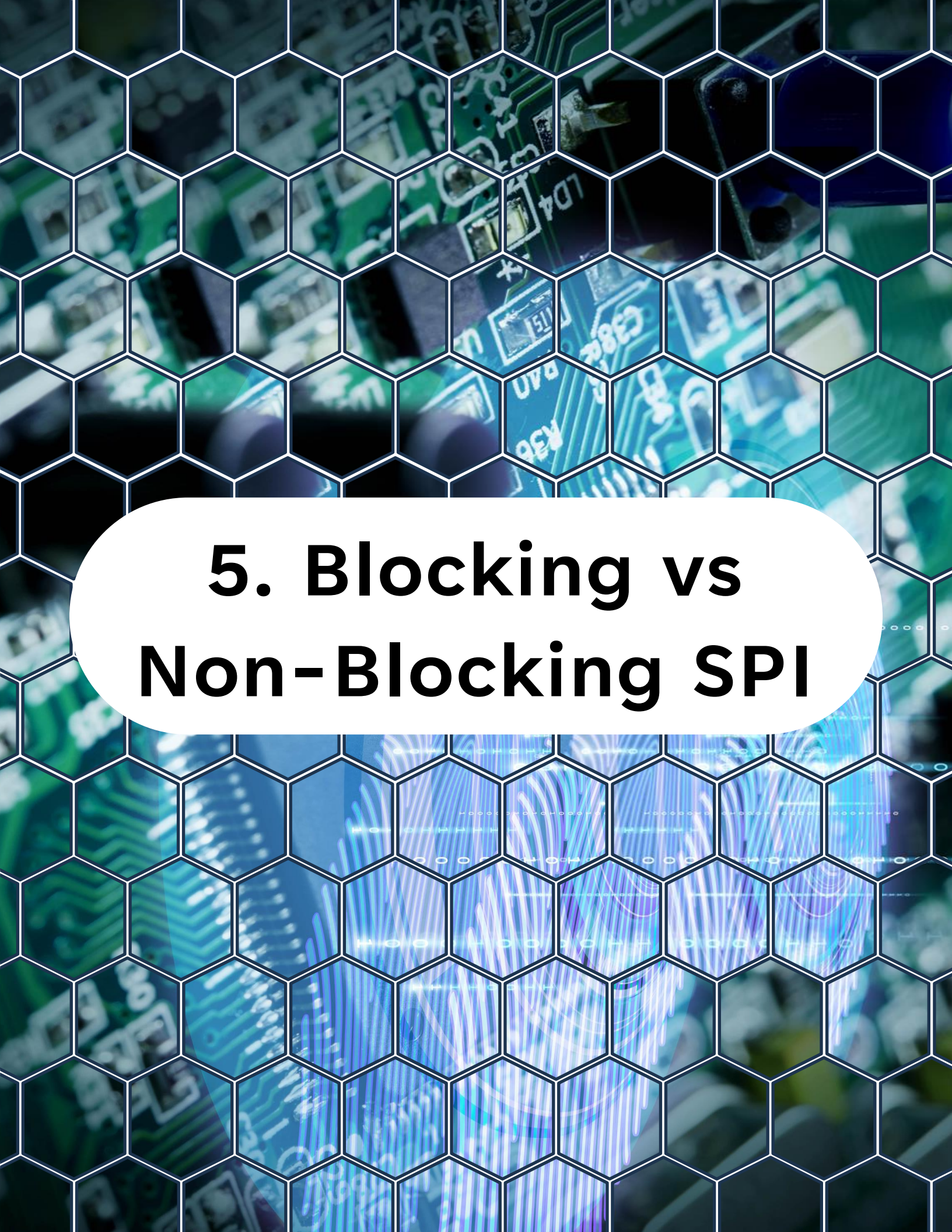
4. Designing a Minimal SPI Driver API

A simple, reusable SPI Master API could look like:



```
1 // Initialize the SPI interface
2 void spi_init(void);
3
4 // Transfer a byte of data over SPI and receive the response
5 uint8_t spi_transfer(uint8_t data);
6
7 // Select a device for communication
8 void spi_select(uint8_t device);
9
10 // Deselect the currently selected device
11 void spi_deselect(uint8_t device);
```

The application never touches SPI registers directly.



5. Blocking vs Non-Blocking SPI

5. Blocking vs Non-Blocking SPI

Blocking SPI

- Wait until transfer completes
- Simpler
- Good for small transfers
- CPU busy during transfer

Non-Blocking SPI

- Interrupt-driven
- Uses state machine
- Allows multitasking
- More complex

For many embedded systems without RTOS, blocking SPI is acceptable if transfers are short. But long transfers (like display refresh) benefit from non-blocking design.



6. Adding Timeout Protection

6. Adding Timeout Protection

Never assume hardware will always respond.

Add timeout to your transfer loop.

Improved Blocking Transfer with Timeout

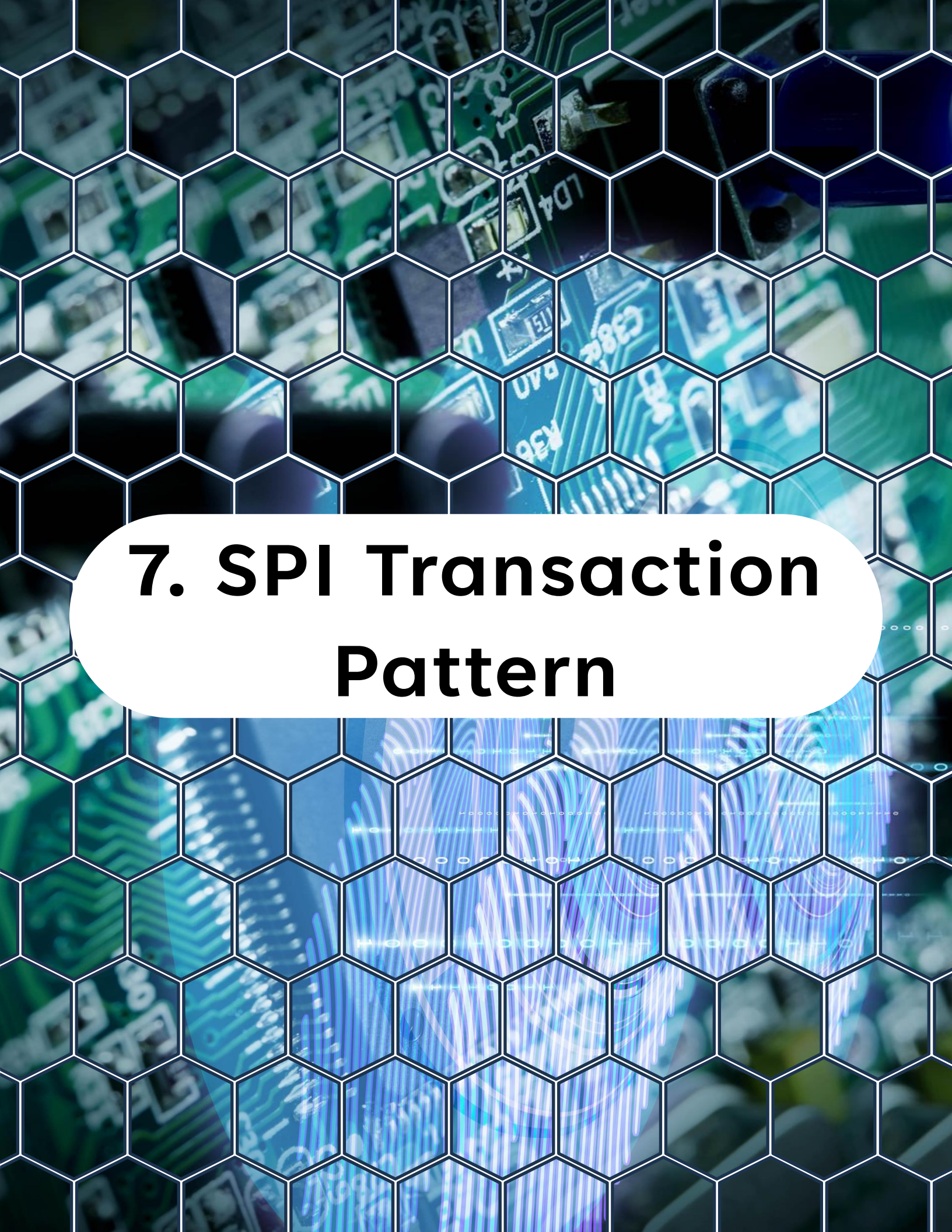
```
1 static uint8_t spi0_transfer_timeout(uint8_t tx, uint32_t timeout)
2 {
3     // Start the SPI transfer by writing to the DATA register
4     SPI0.DATA = tx;
5
6     // Wait for the transfer to complete or timeout
7     while ((SPI0.INTFLAGS & SPI_IF_bm) == 0)
8     {
9         if (timeout == 0)
10        {
11            return 0xFF; /* Timeout error value */
12        }
13        timeout--;
14    }
15
16    // Clear the interrupt flag by writing a 1 to it
17    SPI0.INTFLAGS = SPI_IF_bm;
18
19    // Return the received data from the DATA register
20    return SPI0.DATA;
21 }
```

6. Adding Timeout Protection

Timeout prevents infinite blocking if:

- Slave disconnected
- Clock misconfigured
- SPI peripheral disabled

This is real-world defensive programming.



7. SPI Transaction Pattern

7. SPI Transaction Pattern

Instead of manually asserting and deasserting CS everywhere, create a transaction function.

Transaction Example

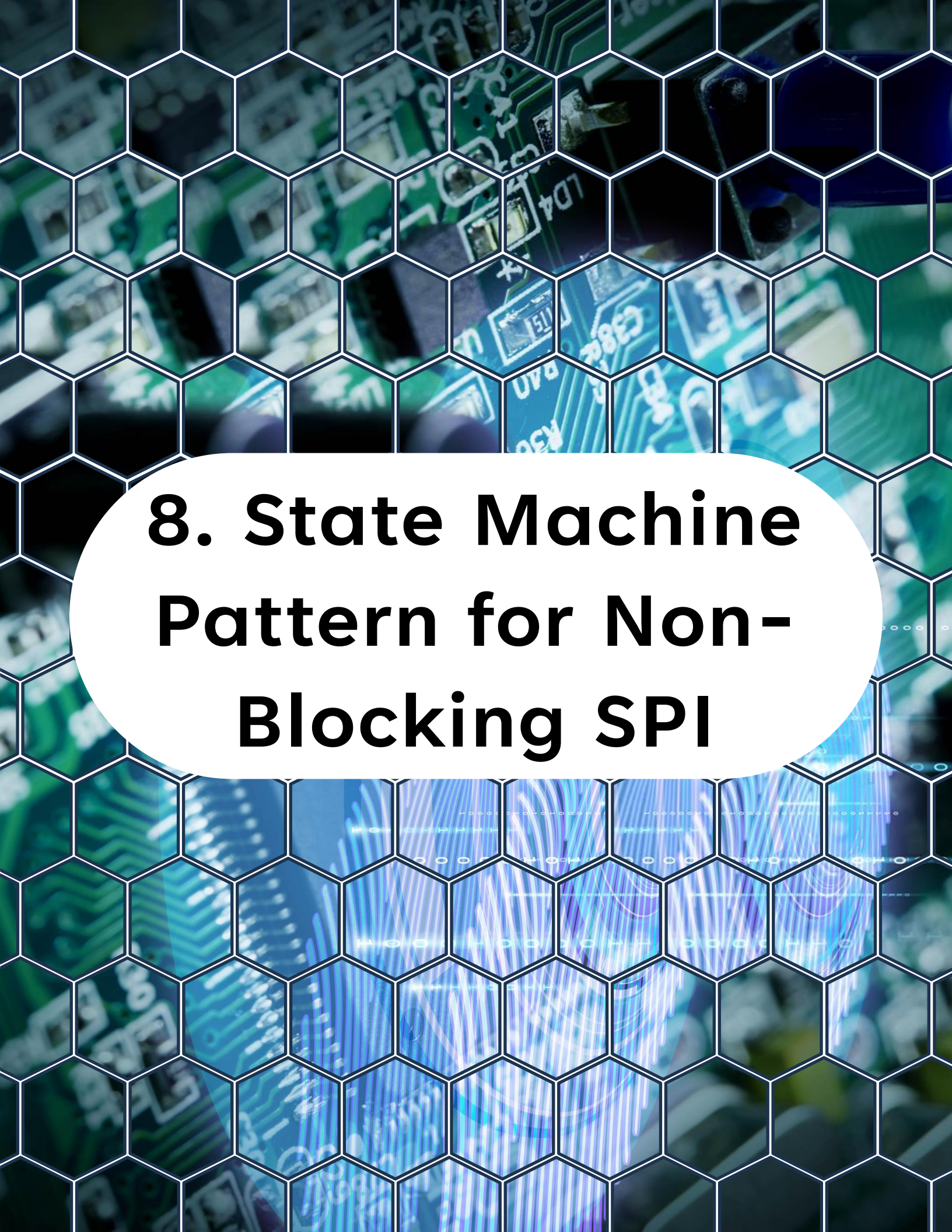


```
1 static void spi0_transaction(const uint8_t *tx,
2                             uint8_t *rx,
3                             uint16_t length)
4 {
5     // Assert the chip select line to start the transaction
6     spi0_cs_assert();
7
8     // Transfer each byte in the transaction
9     for (uint16_t i = 0; i < length; i++)
10    {
11        // Transfer a byte over SPI. If tx is NULL,
12        // send 0xFF (common for SPI reads)
13        uint8_t r = spi0_transfer_byte(tx ? tx[i] : 0xFF);
14
15        if (rx)
16        {
17            rx[i] = r;
18        }
19    }
20
21    // Deassert the chip select line to end the transaction
22    spi0_cs_deassert();
23 }
```

7. SPI Transaction Pattern

Benefits:

- CS handling centralized
- Cleaner device driver code
- Easier debugging



8. State Machine Pattern for Non- Blocking SPI

8. State Machine Pattern for Non-Blocking SPI

For more advanced systems, use a state machine.

States:

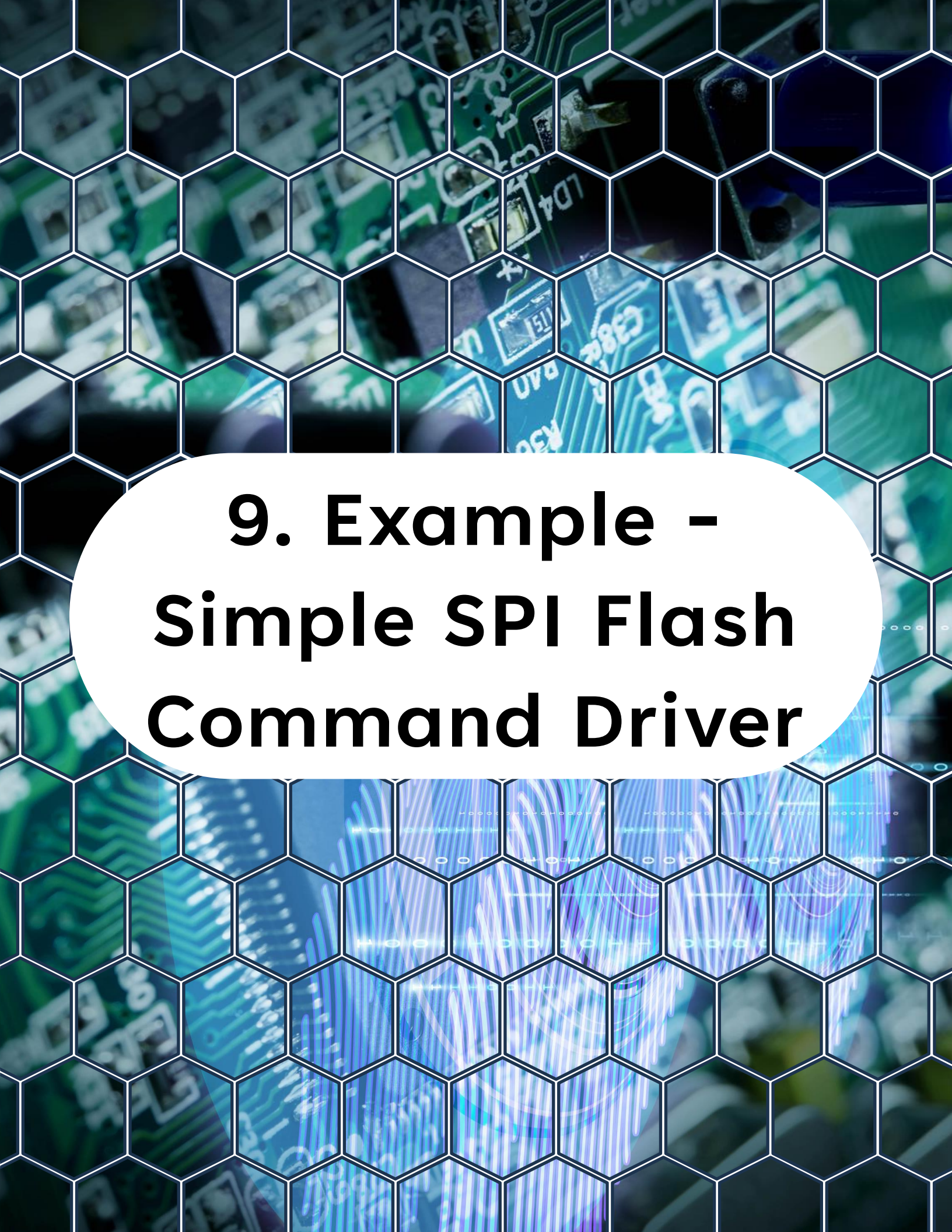
- IDLE
- ASSERT_CS
- TRANSFER
- COMPLETE
- ERROR

ISR handles byte-by-byte transfer.

Main loop checks completion flag.

This pattern allows:

- Long transfers without blocking
- Cooperative multitasking
- Cleaner integration with scheduler (Day 9 concepts)

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white oval contains the title text in a bold, black, sans-serif font.

9. Example - Simple SPI Flash Command Driver

9. Example - Simple SPI Flash Command Driver

Let's simulate a device-level function.

Example: Read JEDEC ID (common SPI flash command 0x9F)

```
1  uint32_t spi_flash_read_id(void)
2  {
3      // Command to read the flash ID (0x9F) followed
4      // by three dummy bytes
5      uint8_t tx[4] = {0x9F, 0xFF, 0xFF, 0xFF};
6
7      // Buffer to store the response from the flash memory
8      uint8_t rx[4] = {0};
9
10     // Send the command and read the response
11     spi0_transaction(tx, rx, 4);
12
13     // Combine the three bytes of the response
14     // into a single 32-bit value
15     return ((uint32_t)rx[1] << 16)
16           | ((uint32_t)rx[2] << 8)
17           | ((uint32_t)rx[3]);
18 }
```

9. Example - Simple SPI Flash Command Driver

Notice:

- Device driver does not access SPI registers
- It only uses transaction API
- Clean separation

This is scalable architecture.

The background of the slide is a close-up photograph of a green printed circuit board (PCB) with various electronic components and traces. Overlaid on this image is a white hexagonal grid pattern. In the center of the slide, there is a white rounded rectangle containing the title text.

10. Handling Multiple SPI Devices

10. Handling Multiple SPI Devices

If you have multiple slaves:

- Each slave has its own **CS** pin
- All share **MOSI**, **MISO**, **SCK**

Create a device selector:



```
1 typedef enum
2 {
3     SPI_DEV_FLASH,
4     SPI_DEV_SENSOR,
5     SPI_DEV_DISPLAY
6 } spi_device_t;
```

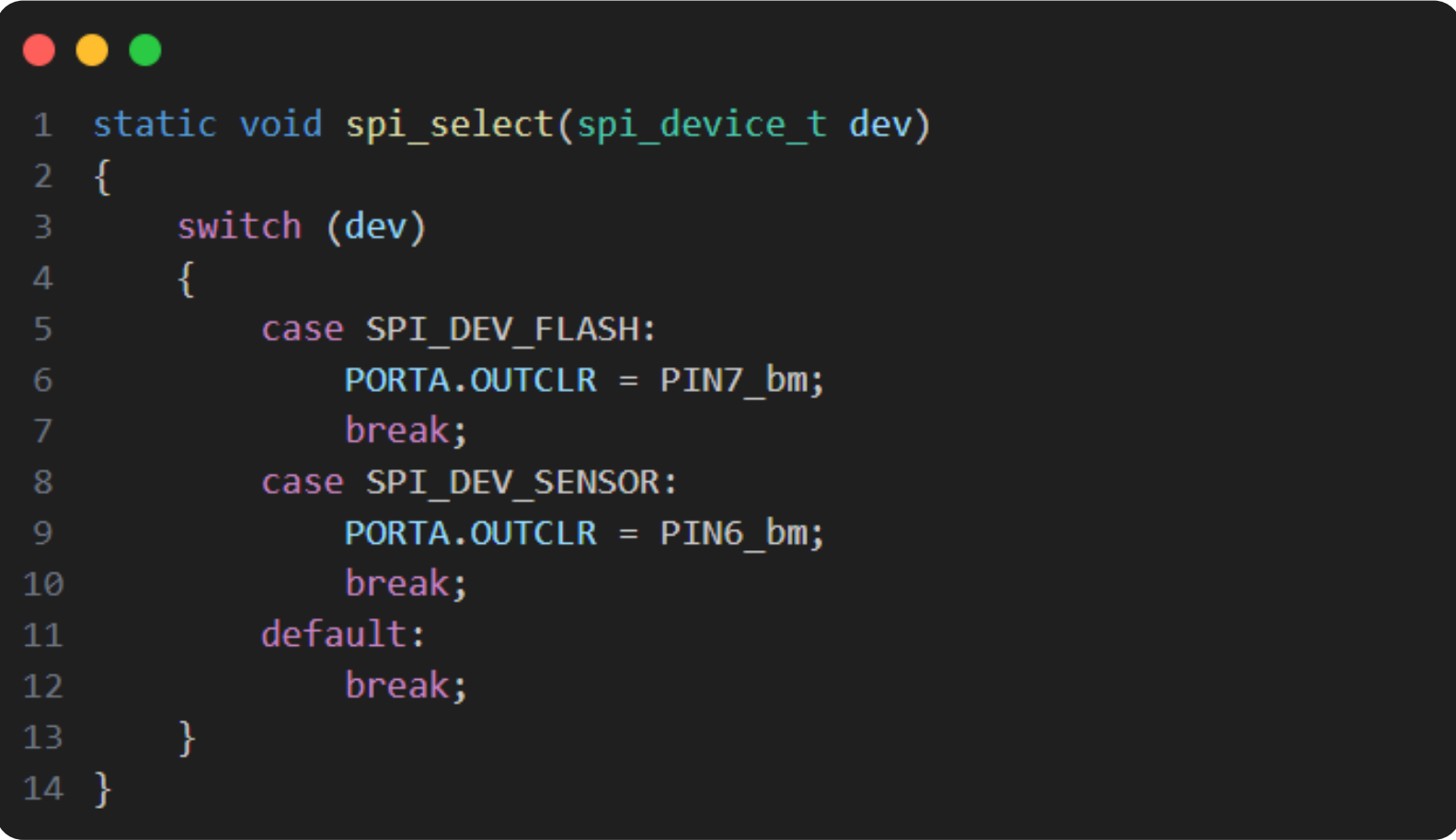
Then in select function:



```
1 static void spi_select(spi_device_t dev)
2 {
3     switch (dev)
4     {
5         case SPI_DEV_FLASH:
6             PORTA.OUTCLR = PIN7_bm;
7             break;
8         case SPI_DEV_SENSOR:
9             PORTA.OUTCLR = PIN6_bm;
```

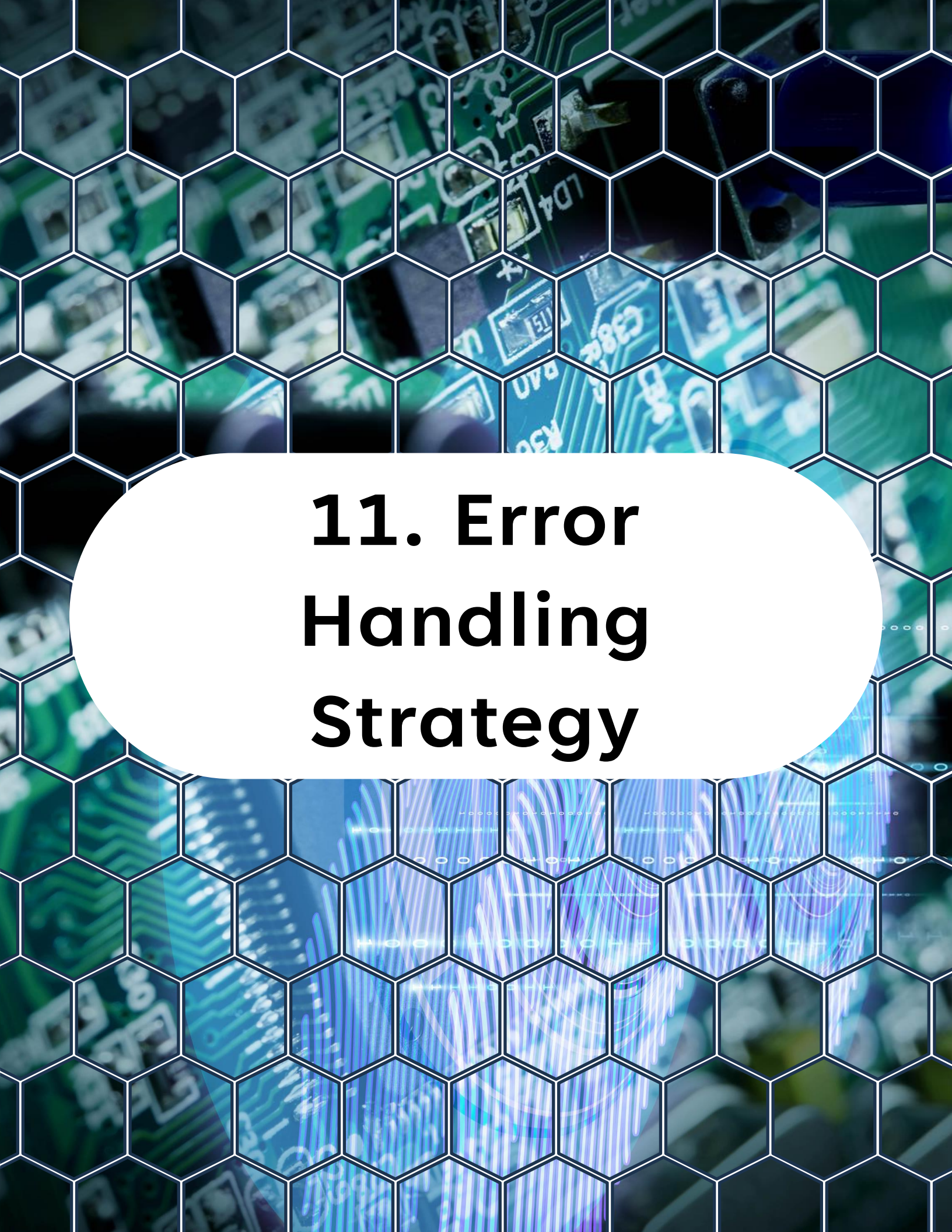
10. Handling Multiple SPI Devices

Then in select function:



```
1 static void spi_select(spi_device_t dev)
2 {
3     switch (dev)
4     {
5         case SPI_DEV_FLASH:
6             PORTA.OUTCLR = PIN7_bm;
7             break;
8         case SPI_DEV_SENSOR:
9             PORTA.OUTCLR = PIN6_bm;
10            break;
11        default:
12            break;
13    }
14 }
```

Never hardcode chip select everywhere in application code.



11. Error Handling Strategy

11. Error Handling Strategy

Real SPI drivers should handle:

- Timeout
- Buffer overflow
- Invalid parameters
- Device not responding

You can add:

- Return status codes
- Error flags
- Retry mechanisms

Example:



```
1 typedef enum
2 {
3     SPI_OK,
4     SPI_TIMEOUT,
5     SPI_ERROR
```

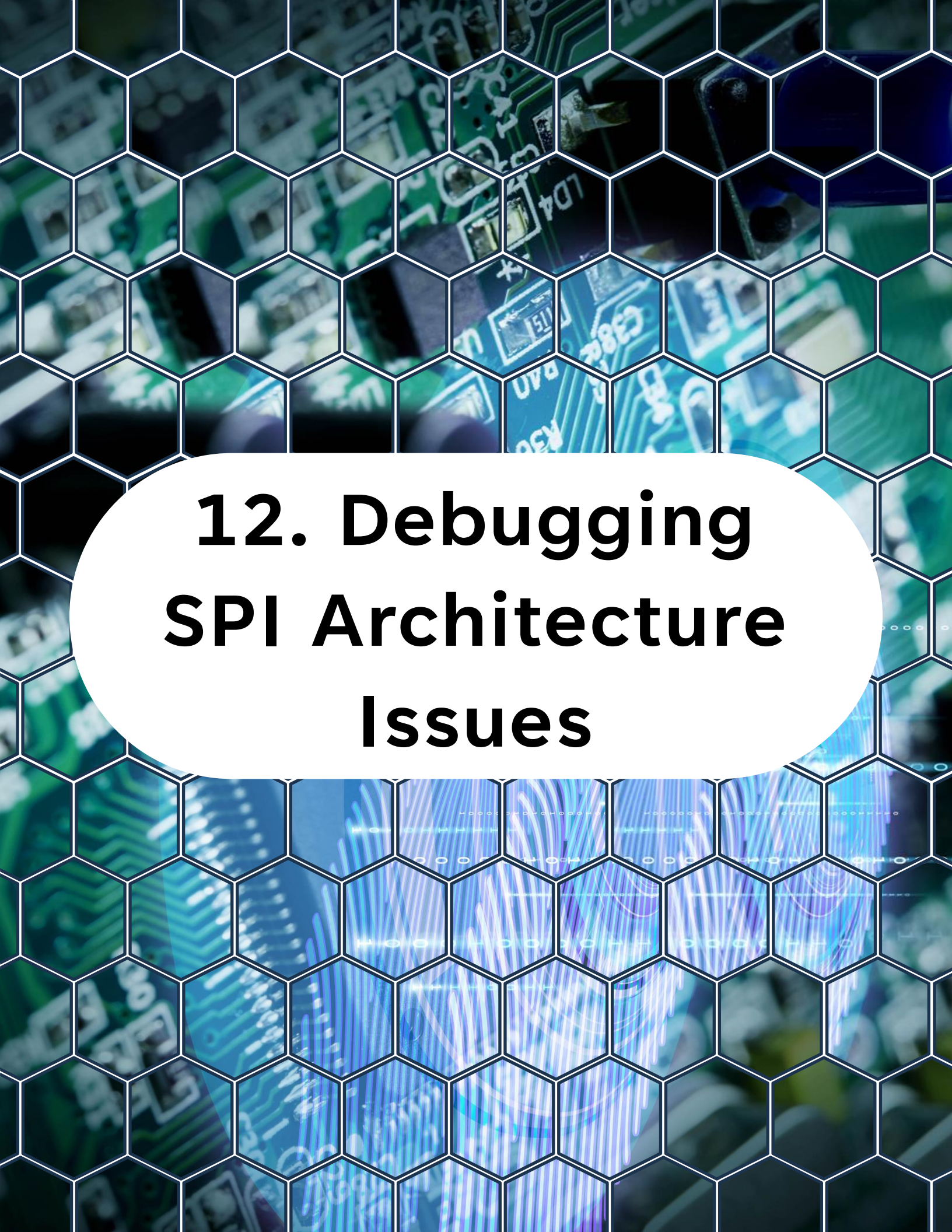
11. Error Handling Strategy

Example:



```
1 typedef enum
2 {
3     SPI_OK,
4     SPI_TIMEOUT,
5     SPI_ERROR
6 } spi_status_t;
```

Returning structured status makes systems robust.



12. Debugging SPI Architecture Issues

12. Debugging SPI Architecture Issues

If architecture is wrong, you will see:

- CS toggling mid-transfer
- Mixed transactions
- Device confusion
- Data corruption

Check:

- Only one device selected at a time
- CS stable during entire transfer
- Correct SPI mode per device
- No concurrent access without arbitration

Architecture prevents these bugs.



13. What You Learned Today

13. What You Learned Today

By the end of **Day 14**, you now understand:

- How to structure SPI drivers properly
- Blocking vs non-blocking patterns
- How to implement timeout protection
- Transaction-based SPI design
- Device abstraction patterns
- Multi-slave handling

This moves you from demo-level SPI to production-level SPI.



14. What Comes Next

14. What Comes Next

Day 15 - I2C (TWI) Fundamentals

Next you will:

- Understand open-drain bus architecture
- Configure TWI in Master mode
- Perform I2C read/write transactions
- Compare I2C vs SPI in real design decisions

From here on, your firmware can communicate with almost any peripheral.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>